КНИГА 3

Архитектура нейросетей и трансформеров

Как на самом деле устроены современные языковые модели

Уровень: Средний — Продвинутый

Библиотека ИИ · 5 книг · все уровни и категории

1. Искусственный нейрон и функции активации

Средний

Базовая единица нейронной сети — искусственный нейрон: он принимает числа на вход, умножает каждое на свой вес, складывает результаты со смещением (bias), и передаёт сумму через нелинейную функцию активации. Без нелинейности сеть из множества слоёв математически эквивалентна одному линейному слою и не может выучивать сложные закономерности.

```
output = activation(w1*x1 + w2*x2 + ... + wn*xn + bias)
```

Популярные функции активации

```
ReLU(x) = max(0, x) # просто и быстро, стандарт по умолчанию
GELU(x) = x * Phi(x) # плавный вариант ReLU, используется в трансформерах
Sigmoid(x) = 1 / (1 + e^(-x)) # сжимает в диапазон (0, 1)
```

2. Прямой проход, функция потерь и обратное распространение

Продвинутый

Обучение нейросети состоит из повторяющегося цикла из трёх шагов: прямой проход (forward pass) вычисляет предсказание сети; функция потерь (loss function) измеряет, насколько предсказание отличается от правильного ответа; обратное распространение ошибки (backpropagation) вычисляет, как изменить каждый вес, чтобы уменьшить эту ошибку.

```
for epoch in range(num_epochs):
    for batch in training_data:
        predictions = model.forward(batch.inputs)
        loss = loss_function(predictions, batch.targets)
        gradients = loss.backward() # обратное распространение
        optimizer.step(gradients) # обновление весов
        optimizer.zero_grad()
```

Обратное распространение использует правило цепочки (chain rule) из математического анализа, чтобы эффективно вычислить вклад каждого веса в итоговую ошибку, двигаясь от выходного слоя обратно к входному.

3. Ограничения рекуррентных сетей (RNN)

Средний

До трансформеров основным инструментом обработки текста были рекуррентные сети (RNN) и их варианты (LSTM, GRU). Они обрабатывают текст последовательно, слово за словом, передавая «скрытое состояние» от одного шага к следующему. У этого подхода два фундаментальных ограничения:

- Последовательность вычислений — нельзя обработать слово 50 до того, как обработано слово 49, что делает обучение медленным и плохо распараллеливаемым на современном оборудовании.
- Затухающий контекст — информация из начала длинного текста постепенно «размывается» к концу последовательности, из-за чего модель хуже учитывает дальние зависимости.

4. Механизм внимания (Attention)

Продвинутый

Внимание решает обе проблемы RNN: вместо последовательной передачи состояния, каждый токен напрямую «смотрит» на все остальные токены одновременно и решает, какие из них наиболее важны для его понимания. Это делается через три проекции каждого токена: Query (запрос), Key (ключ) и Value (значение).

```
Attention(Q, K, V) = softmax( (Q * K^T) / sqrt(d_k) ) * V
```

Интуиция: Query токена — это «что я ищу», Key каждого токена — это «что я предлагаю», их скалярное произведение даёт оценку релевантности, softmax превращает оценки в веса (в сумме дающие 1), а взвешенная сумма Value даёт итоговое представление токена с учётом контекста.

Многоголовое внимание (Multi-Head Attention)

На практике внимание вычисляется параллельно несколько раз («голов») с разными обученными проекциями — это позволяет модели одновременно отслеживать разные типы связей (например, грамматические и смысловые) в одном и том же тексте.

5. Блок трансформера целиком

Продвинутый

Полный трансформер — это стопка одинаковых по структуре блоков, каждый из которых состоит из слоя внимания и полносвязной сети прямого распространения (feed-forward), с нормализацией и «остаточными соединениями» (residual connections) вокруг каждого подслоя.

```
Вход (эмбеддинги токенов + позиционная информация)
   |
   v
[ Multi-Head Self-Attention ]
   | + остаточное соединение, затем нормализация
   v
[ Feed-Forward Network (полносвязный слой) ]
   | + остаточное соединение, затем нормализация
   v
Выход блока --> вход следующего такого же блока
   |
  ... (повторяется N раз, N = десятки слоёв)
   v
Финальный слой --> вероятности следующего токена
```

6. Токенизация: почему модели не считают буквы

Средний

Модель никогда не видит отдельные буквы напрямую — текст сначала разбивается на токены алгоритмом вроде BPE (Byte Pair Encoding) или SentencePiece. Токен может быть целым словом, частью слова или даже несколькими символами, в зависимости от того, как часто эта последовательность встречалась в обучающих данных.

```
Текст: "токенизация"
Токены: ["ток", "ениз", "ация"] # пример разбиения, точное деление зависит от словаря
```

Именно поэтому модели часто ошибаются в задачах вроде «сколько букв «р» в слове «корабль»» — они оперируют токенами, а не отдельными буквами, и не всегда могут «заглянуть внутрь» токена.

7. Эмбеддинги: слова как числа

Средний

Эмбеддинг — это представление токена в виде вектора чисел (обычно от нескольких сотен до нескольких тысяч измерений), устроенное так, что близкие по смыслу слова оказываются близко друг к другу в этом многомерном пространстве. Классический иллюстративный пример из ранних моделей word2vec показывает, что с векторами можно производить «арифметику смысла»:

```
вектор("король") - вектор("мужчина") + вектор("женщина") ~= вектор("королева")
```

8. Предобучение и дообучение: стоимость и цель

Продвинутый

Предобучение (pre-training)

Обучение модели «с нуля» на огромном корпусе текста с общей задачей предсказания следующего токена. Требует огромных вычислительных ресурсов (недели или месяцы на тысячах GPU/TPU) и денежных затрат, измеряемых миллионами долларов для моделей передового уровня.

Дообучение (fine-tuning)

Донастройка уже предобученной модели на значительно меньшем, специализированном наборе данных — например, на парах «вопрос-хороший ответ» — чтобы изменить стиль поведения или добавить навык, не обучая модель заново. Требует на порядки меньше ресурсов, чем предобучение.

9. RLHF: обучение с подкреплением на основе обратной связи людей

Продвинутый

RLHF (Reinforcement Learning from Human Feedback) — этап после базового дообучения, на котором модель учится давать ответы, которые люди-оценщики предпочитают, а не просто статистически вероятные. Упрощённая схема: люди сравнивают пары ответов модели и отмечают, какой лучше -> на этих сравнениях обучается отдельная «модель вознаграждения» (reward model) -> основная модель дообучается методами обучения с подкреплением так, чтобы максимизировать оценку этой модели вознаграждения. Именно этот этап во многом отвечает за то, что современные ассистенты следуют инструкциям и избегают вредных ответов значительно лучше, чем просто предобученная модель.

10. Параметры генерации: температура, top-k, top-p

Продвинутый

На каждом шаге генерации модель выдаёт распределение вероятностей по всем возможным следующим токенам. Параметры сэмплирования определяют, как именно выбирается итоговый токен из этого распределения.

Параметр	Что делает	Низкое значение	Высокое значение
Температура	Насколько «уверенно» выбирается токен	Детерминированно, предсказуемо	Разнообразно, иногда хаотично
Top-k	Рассматривать только k самых вероятных токенов	Мало вариантов (строже)	Много вариантов (разнообразнее)
Top-p (nucleus)	Рассматривать наименьший набор токенов с суммарной вероятностью p	Узкий, надёжный набор	Широкий, более творческий набор

11. Законы масштабирования (Scaling Laws)

Продвинутый

Исследования масштабирования (начиная с работы OpenAI 2020 года и продолженные в работе DeepMind «Chinchilla» 2022 года) показали предсказуемую связь между размером модели, объёмом обучающих данных, вычислительным бюджетом и итоговым качеством модели. Ключевой практический вывод Chinchilla: многие ранние модели были «недообучены» относительно своего размера — для оптимального результата при заданном вычислительном бюджете нужно пропорционально увеличивать не только число параметров, но и объём обучающих данных.

12. Архитектурные ограничения трансформеров

Продвинутый

- Квадратичная стоимость внимания — вычислительные затраты на механизм внимания растут пропорционально квадрату длины последовательности, что ограничивает размер контекстного окна и делает очень длинный контекст дорогим.
- Нет настоящей памяти между сессиями — модель не обучается и не запоминает что-либо после разговора; каждая новая сессия начинается с тех же зафиксированных весов, если не подключена внешняя система памяти.
- Замороженные веса после обучения — в обычном режиме работы (инференс) веса модели не меняются; любое «обучение на лету» требует отдельных специальных механизмов, а не является свойством базовой архитектуры.
- Высокая стоимость вычислений — как обучение, так и инференс моделей передового уровня требуют значительных затрат на электроэнергию и специализированное оборудование (GPU/TPU).

Важно понимать. Эти ограничения — не недостатки конкретных моделей, а свойства самой архитектуры трансформера в её нынешнем виде. Исследования альтернативных архитектур (линейное внимание, state-space модели и другие) ведутся именно с целью обойти эти ограничения.